

Simulação Computacional de um Buraco Negro com OpenGL Legacy

Denilso Bernardo Nunes da Silva¹

¹Bacharelado em Ciência da Computação
Universidade Federal do Ceará (UFC)
Ceará – CE – Brasil

denilsobn@alu.ufc.br

Abstract. *This work presents a simulation of a black hole using Legacy OpenGL. Due to the computational impossibility of calculating real-time light trajectories (relativistic ray tracing) in older graphical pipelines, the simulation relies on particle systems and post-processing texture mapping techniques to emulate gravitational lensing and Doppler beaming. Mathematical models such as Kepler's laws and spherical-to-cartesian coordinate transformations are used to govern the accretion disk and the procedural starfield.*

Resumo. *Este trabalho apresenta uma simulação de um buraco negro utilizando OpenGL Legacy. Devido à impossibilidade computacional de calcular trajetórias reais da luz com ray tracing relativístico em tempo real nesse pipeline gráfico, a simulação utiliza sistemas de partículas e técnicas de mapeamento de textura para emular a lente gravitacional e o efeito Doppler. Modelos matemáticos como as leis de Kepler e conversões de coordenadas esféricas para cartesianas ditam o disco de acreção e a geração procedural de estrelas.*

1. Introdução e Teoria do Buraco Negro

Simular a física de um buraco negro com exatidão requer o cálculo de geodésicas da luz curvadas pela gravidade extrema, algo que exige algoritmos de *ray tracing* não lineares de alto custo. Como o OpenGL Legacy não comporta esse tipo de cálculo em tempo real de forma nativa, a solução adotada foca em uma aproximação visual convincente através de matemática de vetores e de renderização.

O ambiente simula dois componentes astronômicos principais:

- **O Horizonte de Eventos e a ISCO:** O raio do buraco negro é definido e, logo acima dele, encontra-se a ISCO (*Innermost Stable Circular Orbit* - Órbita Circular Estável Mais Interna). Na física real, a ISCO é o limite onde a matéria ainda consegue orbitar sem cair para o horizonte de eventos.
- **O Disco de Acreção:** Formado por gases e poeira espiralando a velocidades altas em torno do buraco negro.

A singularidade do Buraco Negro nesse projeto é representada por uma esfera de cor preta, situada no ponto (0, 0, 0). A função `timer()` é chamada a cada 16ms (o que corresponde a aproximadamente 60Hz), instruindo o OpenGL a desenhar o próximo quadro.

2. Geração de Estrelas de Fundo e Coordenadas Esféricas

O código para gerar as estrelas está na função `initStars()`. Para o cenário espacial de fundo, é criado um campo de estrelas procedural. O uso do sistema de partículas recomenda introduzir aleatoriedade para conferir um aspecto orgânico. Por isso, a variação do brilho de cada estrela é calculada de forma aleatória, criando um efeito de profundidade visual.

Para posicionar essas estrelas de forma homogênea em uma esfera ao redor do observador, o código converte as Coordenadas Esféricas em Coordenadas Cartesianas 3D (X, Y, Z). Dada uma distância R , o ângulo azimutal θ e o ângulo zenital ϕ , as posições são dadas por:

$$X = R \cdot \sin(\phi) \cdot \cos(\theta)$$

$$Y = R \cdot \sin(\phi) \cdot \sin(\theta)$$

$$Z = R \cdot \cos(\phi)$$

Essa abordagem evita o acúmulo de estrelas nos polos que ocorreria em uma distribuição cilíndrica e se aproxima de um cenário real.

3. Ciclo de Vida das Partículas e Lei de Kepler

Cada partícula é definida por uma estrutura de dados que armazena características como raio, altura, ângulo e suas respectivas cores RGB. O raio, a altura e o ângulo são utilizados para determinar as coordenadas homogêneas abordadas anteriormente, permitindo que as partículas sejam geradas e distribuídas corretamente ao redor do buraco negro.

O disco de acreção é construído por partículas dinâmicas. O ciclo de vida de cada partícula obedece a três fases:

1. **Nascimento:** As partículas nascem na função `initParticulas()`. As partículas são geradas distribuídas entre a ISCO e o raio mais externo (`DISK_OUTER_RADIUS`), com ângulos iniciais aleatórios e uma leve variação de altura para dar volume ao disco.
2. **Vida:** A função `timer()` atua como uma dimensão do tempo. A velocidade de rotação no ângulo de uma partícula segue uma aproximação da Terceira Lei de Kepler, onde a velocidade angular depende do raio: $\omega \propto r^{-3/2}$. Essa velocidade aumenta à medida que a função é chamada. Ao mesmo tempo, o raio diminui gradativamente, emulando a espiral da morte em direção ao horizonte.
3. **Morte e Renascimento:** Quando a partícula cruza a ISCO (`particulas[i].raio < ISCO`), ela é virtualmente engolida. O sistema a reaproveita recriando-a na borda externa do disco com um novo ângulo, fechando o loop de vida.

4. Ordenação de Profundidade

Para que a distorção gravitacional atue corretamente, o buraco negro deve distorcer apenas o que está atrás dele. Para isso, o disco de acreção é renderizado em duas etapas: a parte de trás e a parte da frente.

A matemática para separar o disco utiliza o produto escalar entre o vetor de posição da partícula e o vetor da câmera. O produto escalar calcula a projeção e, consequentemente, revela o cosseno do ângulo entre os vetores.

Na função `drawAccretionDisk()`, quando a operação `dotPlane = (px*camX) + (py*camY) + (pz*camZ)` resulta em um valor positivo, significa que o ângulo entre os vetores está entre 0° e 90° . Ou seja, a partícula se encontra do mesmo lado do plano que a câmera. Valores negativos indicam que a partícula está atrás do plano que corta o buraco negro. Esse cálculo permite um fatiamento no eixo do espaço tridimensional. Isso é visto na figura 1.

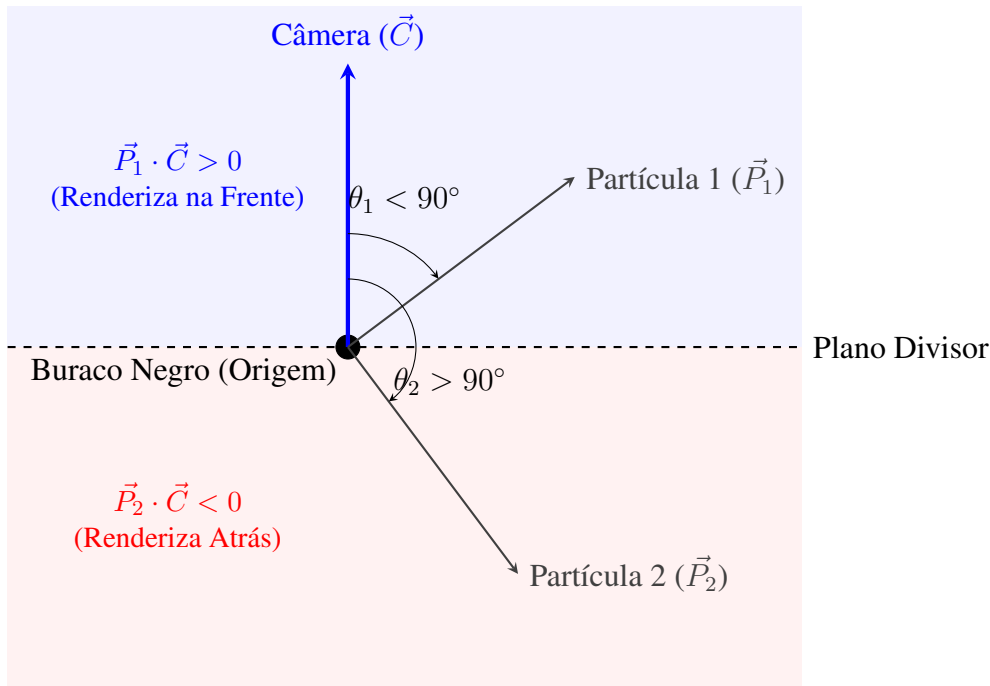


Figura 1. Representação vetorial utilizando o Produto Escalar para determinar a visibilidade das partículas do disco de acreção em relação à câmera.

Outro detalhe acontece quando o `dotProduct > 0.4f`. Tal valor mostra que o vetor da partícula está próximo da câmera e vindo em sua direção. Para criar um efeito *doppler*, o desvio para o azul dessa partícula é aumentado. Isso pode ser observado na simulação.

5. Lente Gravitacional Com Mapeamento de Textura

Como não se pode calcular o trajeto da luz em tempo real para criar a lente gravitacional, utiliza-se uma técnica de espaço de tela. O processo ocorre da seguinte forma:

1. Renderizam-se as estrelas e a parte de trás do disco.
2. A função `glCopyTexImage2D` captura o *framebuffer* e armazena em uma textura.
3. A tela é limpa, e desenha-se uma malha quadriculada de 60 quadros preenchendo a janela.
4. A função `drawDistorcion()` mapeia a textura na tela, causando um efeito de distorção.

5.1. Mapeamento do Quadriculado

Para desenhar o quadriculado ocupando a tela inteira com a textura capturada sem mexer com a configuração de perspectiva da câmera, utilizamos a pilha de matrizes (`glPushMatrix()` e `glPopMatrix()`). O código salva o estado atual e carrega uma projeção ortográfica (`glOrtho`), que mapeia o desenho para o cubo canônico 2D com as coordenadas normalizadas de $[-1, 1]$. Após aplicar a textura distorcida, o `glPopMatrix()` restaura o mundo 3D.

Antes de chamar a função `DrawDistorcion()`, é armazenada a quantidade de quadrados na tela. A variável `square` guarda o tamanho de cada quadrado, dividindo o tamanho da tela no cubo canônico pela quantidade de quadrados. Em seguida, com o `glBegin(GL_QUADS)`, é desenhada a malha quadriculada. Os quadrados são desenhados na tela com cores que não alteram a cor dos objetos. O cálculo de (x, y) acontece de acordo com o mapeamento $[-1, 1]$ da tela.

O cubo canônico está situado no espaço $[-1, 1]$, enquanto a textura está em $[0, 1]$. Para mapear o cubo canônico nas coordenadas na textura, é usada uma transformação de Escala seguida de Translação. O cubo canônico tem tamanho 2, a textura tem tamanho 1, logo, é necessário aplicar uma escala de 0.5. O ponto do cubo canônico é $(-1, -1)$, é necessário transladar para o ponto $(0, 0)$. Isso resulta em uma matriz de transformação aplicada nos pontos (u, v) no código. Cada ponto é multiplicado pelo escalar 0.5 e sofre uma translação de 0.5. Essa transformação pode ser observada na Figura 2.

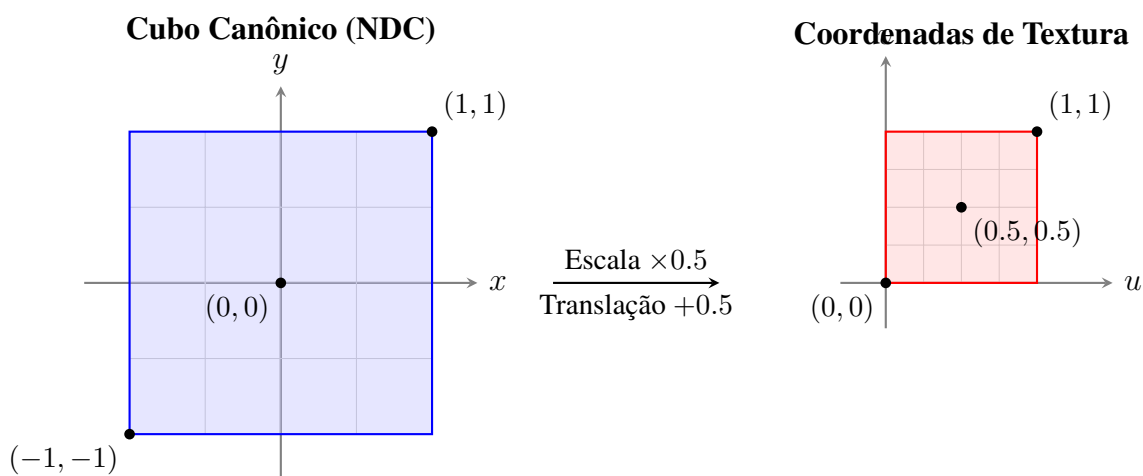


Figura 2. Transformação do espaço de vértices $[-1, 1]$ para o espaço de textura $[0, 1]$.

5.2. O Algoritmo de Distorção

Na função `DrawDistorcion()`, ao desenhar essa malha canônica, a função de distorção calcula a distância r de cada vértice ao centro. É levada em consideração a razão de proporção (*aspect ratio*) da tela, pois a janela pode não ser um quadrado perfeito. Sem esse ajuste, a distorção ficaria achatada e o mapeamento na textura não iria condizer com a tela. Com r calculado, as coordenadas (u, v) da textura recebem um desvio (*shift*), simulando o desvio da luz provocado por uma grande massa. Segundo a Lei da Gravitação Universal, a gravidade diminui com o quadrado da distância. Isso é feito na variável *shift*,

onde $\text{shift} = \text{mass} / (r * r)$. Essa distorção pode ser aumentada ou diminuída alterando a variável mass , representando a massa do buraco negro.

É usado um número fixo para representar qual o limite do raio para acontecer a distorção. Esse número foi uma aproximação do autor, que representaria 10% do raio da tela. Por esse motivo, não é possível implementar um zoom na simulação. Ao final, com as coordenadas (u, v) alteradas de acordo com a distorção, são mapeadas na tela, causando a distorção.

Nas Figuras 3 e 4, é representado um centro na posição $(0.5, 0.5)$ para uma melhor visualização e mostra como essa distorção ocorre. Visualmente, isso implica que, para pintar o pixel atual (x, y) , o OpenGL vai buscar a cor de um pedaço da imagem que está mais próximo do buraco negro (u, v) , criando a ilusão de que as estrelas do fundo foram esticadas para as bordas.

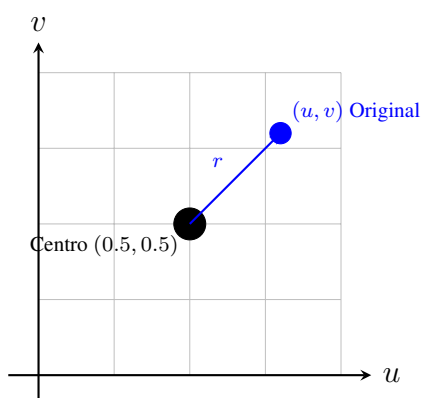


Figura 3. Mapeamento de textura normal, sem interferência gravitacional.

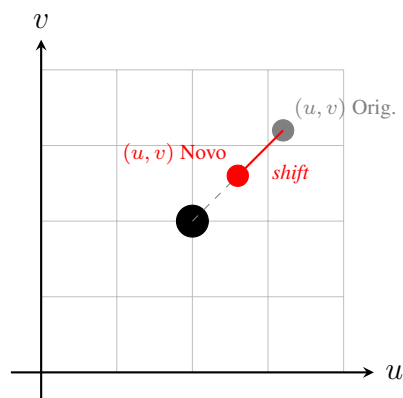


Figura 4. Mapeamento distorcido. A coordenada de leitura é puxada em direção à massa.

A função de redimensionamento atualiza o tamanho da porta de visualização (*viewport*) para que o buraco negro e a lente acompanhem a tela. Porém, como a distorção opera estaticamente nos pixels já renderizados na textura usando limites definidos mencionados anteriormente, dar zoom na câmera alteraria as proporções tridimensionais sem que a malha 2D de distorção soubesse interpretar matematicamente a nova profundidade focal.

6. Conclusão

Esse trabalho demonstra a importância do manuseio e do uso da API legada do OpenGL. Aproximar processos físicos complexos, como o Espaço-Tempo curvo de Einstein e o Efeito Doppler da luz, através de geometria linear, álgebra de vetores, pilhas de matrizes ortográficas e gerenciamento do ciclo de vida das partículas, resulta numa representação interativa eficaz para fins de aprendizagem.